

ETHEREAL · SECURITY REVIEW

Ethereal Round 2.

A security review of Round 2.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Ethereal	Round 2	Guardian	June 11 – June 25, 2025	July 7, 2025

§ 00

Contents.

94 pages — overview, confidence ranking, scope, 58 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 08
	Main Review · 58 findings	P. 12
§ 05	Disclaimer	P. 93
§ 06	About Guardian	P. 94

§ 01

Overview.

Guardian reviewed Round 2 across a single review round. The final report includes 1 Critical, 5 High, 16 Medium, 13 Low, 23 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	meridianxyz/sequencer, meridianxyz/contracts
CLIENT	METHODOLOGY
Ethereal	Smart contract · Manual · Main Review
PROJECT	MAIN REVIEW COMMIT
Round 2	c11cfc5d94ea
PREPARED BY	
Guardian	
AUDIT WINDOW	
June 11 – June 25, 2025	
FINAL REPORT	
July 7, 2025	

1.1 Confidence Ranking

3 / 5

Moderate
Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
■ Critical	1	0	0	0	1
■ High	5	0	1	0	4
■ Medium	16	0	9	0	7
■ Low	13	0	8	0	5
■ Informational	23	0	14	0	9
All severities	58	0	32	0	26

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Round 2 in `meridianxyz/sequencer`, `meridianxyz/contracts`. The scope includes 2,769 source lines across 39 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope 1/2.

FILE	NSLOC	TOTAL
src/ActionHandler.sol	335	431
src/CollateralManager.sol	163	235
src/EtherealProxy.sol	5	8
src/ExchangeConfig.sol	318	462
src/ExchangeGateway.sol	357	535
src/Liquidation.sol	265	389
src/PerpEngine.sol	391	526
src/Verifier.sol	13	17
src/share/Constants.sol	4	6
src/share/DecimalMath.sol	18	36
src/share/Enums.sol	23	32
src/share/Errors.sol	80	206
src/share/RegistryRecords.sol	7	9
src/share/Signatures.sol	11	16
src/share/UserDefinedTypes.sol	2	4
src/external/WUSDe.sol	26	56
src/interface/IActionHandler.sol	103	134
src/interface/ICollateralManager.sol	11	24
src/interface/IExchangeConfig.sol	38	99
src/interface/IExchangeGateway.sol	30	48
src/interface/ILiquidation.sol	28	48
src/interface/IPerpEngine.sol	49	71
src/interface/IPythLazer.sol	3	6
src/interface/IVerifier.sol	4	12
src/lib/storage/Account.sol	92	151
src/lib/storage/AddressRegistry.sol	13	18
src/lib/storage/Deposit.sol	8	14
src/lib/storage/Exchange.sol	45	69
src/lib/storage/FeeCollector.sol	24	34
src/lib/storage/FeeSchedule.sol	31	44
src/lib/storage/Liquidator.sol	14	20
src/lib/storage/PerpPosition.sol	13	20
src/lib/storage/PerpProduct.sol	45	67
src/lib/storage/ProductRegistry.sol	20	29
src/lib/storage/Token.sol	29	42
src/lib/storage/Withdraw.sol	9	15

Files in Scope 2/2.

FILE	NSLOC	TOTAL
src/share/tokens/ERC20Helpers.sol	47	56
src/share/tokens/IERC20Extend.sol	4	8
src/lib/external/pyth/PythLazerLib.sol	91	113

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Deposit Fee Not Transferred	Logical Error	CRITICAL	RESOLVED	p. 12
H-01	WebSocket Rate-Limiter Does Not Handle Exceeded-Quota Errors	Logical Error	HIGH	RESOLVED	p. 13
H-02	Multiple OCO's In Group Filled	DoS	HIGH	RESOLVED	p. 16
H-03	Precision Loss In Close-Only Guard Allows Unintended Exposure Increases	Logical Error	HIGH	RESOLVED	p. 17
H-04	Order Match DoS With Close Only Product	DoS	HIGH	RESOLVED	p. 19
H-05	Silent Precision-Loss in Mark-Price Publishing Path	Logical Error	HIGH	ACKNOWLEDGED	p. 21
M-01	Insecure WebSocket Connection	Connections	MEDIUM	ACKNOWLEDGED	p. 24
M-02	HTTP And WebSocket Requests Resolve To The Load-balancer IP As trustProxy Is false	DoS	MEDIUM	ACKNOWLEDGED	p. 25
M-03	Malicious Subaccount Can Prevent Actions	Logical Error	MEDIUM	ACKNOWLEDGED	p. 28
M-04	Mixed-case Ethereum Addresses Bypass Per-Account Rate-Limit Buckets	DoS	MEDIUM	RESOLVED	p. 29
M-06	Clients Can Never Unsubscribe From Liquidation WebSocket Stream	Logical Error	MEDIUM	RESOLVED	p. 32

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-07	Precision-loss In getProductPriceRange	Logical Error	MEDIUM	ACKNOWLEDGED	p. 34
M-09	linkSignerExpiryDays Default Value Mismatch	Warning	MEDIUM	RESOLVED	p. 37
M-10	safe-array-where-in Plugin Returns Wrong Results For NOT IN ()	Logical Error	MEDIUM	ACKNOWLEDGED	p. 39
M-11	Precision Loss On Funding	Precision	MEDIUM	ACKNOWLEDGED	p. 41
M-12	Rate-limiting Imbalance On POST /order/cancel	Warning	MEDIUM	RESOLVED	p. 42
M-13	Revoked Signer Submits Order With Race Condition	Logical Error	MEDIUM	RESOLVED	p. 44
M-14	Incorrect Cache Refresh Logic in getWithdrawLockout	Logical Error	MEDIUM	ACKNOWLEDGED	p. 45
M-16	Incorrect NULL Comparison in isLiquidated Filter	Logical Error	MEDIUM	RESOLVED	p. 47
M-17	Signer Revoke Prevented With Unfillable Order	DoS	MEDIUM	ACKNOWLEDGED	p. 48
M-18	Engine Fill Cost Precision Loss	Logical Error	MEDIUM	ACKNOWLEDGED	p. 49
M-19	Engine And Sequencer Discrepancy For Unfilled Market Order	Logical Error	MEDIUM	RESOLVED	p. 50
L-01	Typo in capActionsUsingEstimatedGasUsage	Logical Error	LOW	RESOLVED	p. 51
L-01.2	Insufficient Checks In depositOnBehalf	Validation	LOW	RESOLVED	p. 52
L-02	Possible Inaccuracy In Funding Rate Calculation Due to Clamping Mechanism	Warning	LOW	ACKNOWLEDGED	p. 53
L-02.2	No Onchain maxOpenInterestUsd Validation	Validation	LOW	ACKNOWLEDGED	p. 55

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-03.2	Nonce Usage Drift	Warning	LOW	ACKNOWLEDGED	p. 56
L-04.2	Lacking DisableInitializers	Warning	LOW	RESOLVED	p. 57
L-03	Standalone OCO Orders Not Supported	Logical Error	LOW	RESOLVED	p. 58
L-04	Potential Manipulation of Funding Rate via Strategic Order Placement	Validation	LOW	ACKNOWLEDGED	p. 59
L-08	Race Condition in Signer Linking Allows Exceeding Quota	Race Condition	LOW	ACKNOWLEDGED	p. 62
L-09	64-byte Signatures Are Rejected	Validation	LOW	ACKNOWLEDGED	p. 63
L-10	Unbounded Sub-account Creation	Validation	LOW	ACKNOWLEDGED	p. 65
L-11	Precision Rounding in PennyJar Dust Metrics	Warning	LOW	ACKNOWLEDGED	p. 66
L-13	Assymetrical Validations For Liquidations	DoS	LOW	RESOLVED	p. 67
I-01	Non-standard Rate-Limit Headers	Validation	INFORMATIONAL	RESOLVED	p. 68
I-01.2	Price Feed With Non-8 Decimal Exponents	Warning	INFORMATIONAL	RESOLVED	p. 70
I-02	Incorrect Return Property	Typo	INFORMATIONAL	RESOLVED	p. 71
I-02.2	Facets Entry-points Lack a "gateway-only" Guard	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 72
I-03.2	Documentation Typos	Informational	INFORMATIONAL	RESOLVED	p. 73
I-04.2	depositOnBehalf Can Be DoS'ed	Frontrunning	INFORMATIONAL	ACKNOWLEDGED	p. 74
I-05.2	Consider Native Transfer During claimFees	Informational	INFORMATIONAL	RESOLVED	p. 75
I-06.2	wUSDe Deposits Withdrawn As Native	Logical Error	INFORMATIONAL	RESOLVED	p. 76

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-07.2	Referral Gaming	Warning	INFORMATIONAL	RESOLVED	p. 77
I-08.2	Lacking Compatibility With Non-USD Tokens	Warning	INFORMATIONAL	RESOLVED	p. 78
I-09.2	Pure Modifier Used On Storage Loads	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 79
I-03	Number Of Positions Opened Are Not Limited At Smart Contract Level	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 80
I-04	Signer Typo	Typo	INFORMATIONAL	RESOLVED	p. 81
I-06	Stale Oracle Prices Can Be Used For Funding Calculations	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 82
I-07	Information Leakage About Orders	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 83
I-08	Whitelist Enabled During Market Activity	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 84
I-09	OCO Orders Created After Trigger	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 85
I-10	Excluded Signers Should Be Validated	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 86
I-11	Action Batch DoS Resiliency	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 87
I-12	Inconsistent Subaccount Throw Handling	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 88
I-13	Leaves Quantity Naming	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 89
I-14	Table-wide EXCLUSIVE Lock On relayAction	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 90
I-15	Dockerfile Warning	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 92

CRITICAL C-01

RESOLVED

Deposit Fee Not Transferred

CATEGORY Logical Error
LOCATION CollateralManager.sol
REVIEW Main Review

DESCRIPTION

Function `_verifyDeposit` returns the `normalizedNetAmount` which is then transferred to the system by the user during a deposit.

The `normalizedNetAmount` however does not include the `depositFee`. Consider the following scenario:

- (1) Alice requests to deposit 100 USDe
- (2) The token fee is 10 USDe
- (3) A pending deposit is created with amount: 90 USDe, fee: 10 USDe and Alice transfers only 90 USDe
- (4) Alice is credited with 90 USDe and Fee collector accrues 10 USDe of fees on `_handleFinalizeDeposit`
- (5) The funds for the fees do not exist since the fees were never deposited. Rather they are taken out of user's deposited collateral.

RECOMMENDATION

`_verifyDeposit` should return the `normalizedAmount`

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/3a3b67843705f31696441dbd6ea61751>

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/commit/796c0ba738f699f87737820f784a1ae28a5b3f90>

HIGH **H-01**

RESOLVED

WebSocket Rate-Limiter Does Not Handle Exceeded-Quota Errors

CATEGORY Logical Error
LOCATION rate-limit.guard.ts
REVIEW Main Review

DESCRIPTION

```
// src/modules/rate-limit/rate-limit.guard.ts
private async wsResponseHandler(
  client: Socket,
  key: string,
  rateLimiter: ChannelRateLimit,
  points: number
) {
  try {
    await Promise.all([
      rateLimiter.general.consume(key, points),
      rateLimiter.burst.consume(key, points),
    ]);
  } catch (err: unknown) {
    handleRateLimitError(client, err, this.logger); // ← logs / emits, but...
  } // ← ...nothing is thrown
}

// ...later in the same file ...
case 'ws': {
  const socket = execContext.switchToWs().getClient() as Socket;
  await this.wsResponseHandler(socket, /*...*/); // may exceed quota
  return true; // Guard always succeeds
}
```

When the Redis token bucket is empty `rateLimiter.consume()` throws `RateLimiterRes`. The catch block logs and calls `handleRateLimitError()`, but does not re-throw and therefore the Guard reports success (`return true`). Therefore, NestJS proceeds to execute the route handler; the action is sequenced and eventually, put on-chain, the very thing the limiter is meant to prevent.

H-01 WebSocket Rate-Limiter Does Not Handle Exceeded-Quota Errors

CONTINUED

HTTP follows a different path: its catch block does convert `RateLimiterRes` into an HTTP 429, correctly aborting the request. The inconsistency makes WebSocket a potential attack vector.

Practical exploitation

```
const socket = io('wss://sequencer.example.com');
while (true) {
  socket.emit('submitOrder', orderPayload); // send thousands per second
}
```

Even after the quota is exhausted the messages continue to be accepted; at most the attacker sees non-fatal "rate-limit" events in the console.

This way, an attacker can bypass account or IP quotas and flood the matching engine. The sequencer still verifies signatures, writes into the database and calls `processActions` on-chain, consuming CPU and relayer gas.

— RECOMMENDATION

Consider failing the Guard when quota is exceeded:

```
private async wsResponseHandler(/*...*/): Promise<void> {
  await Promise.all([
    rateLimiter.general.consume(key, points),
    rateLimiter.burst.consume(key, points),
  ]);
}
```

and in the case 'ws' block:

```
try {
  await this.wsResponseHandler(/*...*/);
  return true;
} catch (err) {
  handleRateLimitError(socket, err, this.logger);
  return false; // Prevents handler execution
}
```

Optionally, disconnect or "soft-ban" the socket inside `handleRateLimitError()` to discourage reconnect-spam:

```
function handleRateLimitError(client: Socket, err: unknown, logger: Logger) {
  if (err instanceof RateLimiterRes) {
```

H-01 WebSocket Rate-Limiter Does Not Handle Exceeded-Quota Errors

CONTINUED

```
client.emit('error', { code: 429, retryAfter: err.msBeforeNext });
client.disconnect(true); // immediate cut-off
}
// ...
}
```

— RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/sequencer/commit/b8f4c9704c2f6580e28ebfe1dadcf1f08fae335>

HIGH **H-02**

RESOLVED

Multiple OCO's In Group Filled

CATEGORY	DoS
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

As part of the conditional order feature, an OTO (one trigger other) can have the "other" be another conditional order, such as an OCO (one cancels other).

The OCO can then have an "other" be another OCO and so on. If the OTO executes, the first OCO is expected to execute. If the first OCO is executed, the second, third, and so on conditional orders are expected to be cancelled.

However, when Nekuti encounters a chain of OCO's and one order in the group obtained a fill, it does not cancel all the remaining OCO's, which goes against Nekuti documentation that "If any order in the group is partially filled, all other orders in the group are canceled immediately"

— RECOMMENDATION

Prevent chains of conditional orders from being submitted by users.

PROOF OF CONCEPT <https://github.com/GuardianOrg/meridian-sequencer/pull/1>

— RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/sequencer/commit/3c7b44ed27cfab7880a9d3608e2d6f0413e2fa03>

HIGH **H-03**

RESOLVED

Precision Loss In Close-Only Guard Allows Unintended Exposure Increases

CATEGORY Logical Error
LOCATION order.service.ts
REVIEW Main Review

DESCRIPTION

The method `OrderService.verifyOrderReducesPosition` is supposed to block any new exposure on products marked `CLOSE_ONLY`. Instead, it converts both the stored position size and the incoming order quantity from full-precision `bigint` into JavaScript number (IEEE-754 double) before checking:

```
const size = Number(position.size); // bigint → double
const sizeDelta = Number(Gwei.fmt(quantity)); // bigint → string → double
const isNotReducingPosition =
  side === OrderSide.BUY
  ? size > 0 || size + sizeDelta > 0
  : size < 0 || size - sizeDelta < 0;
```

IEEE-754 doubles only reliably represent integers up to $2^{53}-1$ ($\approx 9.007 \times 10^{15}$). Because all quantities are stored in gwei (10^{-9} native units), the moment your position or order exceeds roughly 9 million native units, the conversion to number silently rounds, hiding the true delta. A crafted `BUY` order can actually flip a short into a net-long position, yet `size + sizeDelta` in double-precision still appears non-positive, so the guard mistakenly permits it.

Example:

```
const positionSize = -9007199254740999n;
const buyQuantity = 9007199254741000n
const size = Number(positionSize); // -9007199254741000
const sizeDelta = Number(buyQuantity); // 9007199254741000
// size + sizeDelta = 0 -- Position appears reduced after precision-loss
// positionSize + buyQuantity = 1 -- Position actually increased
```

H-03 Precision Loss In Close-Only Guard Allows Unintended Exposure Increases

CONTINUED

— RECOMMENDATION

Perform the comparison using full-precision integers instead of number casts. Replace the two lines above with:

```
const size = BigInt(position.size); // keep as bigint
const sizeDelta = quantity; // already bigint
const wouldIncreaseExposure =
  side === OrderSide.BUY
    ? size + sizeDelta > 0n
    : size - sizeDelta < 0n;
if (wouldIncreaseExposure) {
  throw new BadRequestException('Product ${ticker} is close only');
}
```

— RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/sequencer/commit/ea099dc85ab02670503816f3d8aff3845eb85dc8>

HIGH **H-04**

RESOLVED

Order Match DoS With Close Only Product

CATEGORY	DoS
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

Function `_verifyNotReduceOnlyOrProductCloseOnly` is used when settling the positions on-chain to prevent an increase order being processed when the product is set to `CLOSE_ONLY` mode.

Although Sequencer validations such as below attempt to prevent this revert from being triggered on-chain, it does not entirely prevent it due to pending increase limit orders:

```
if (product.status === ProductStatus.CLOSE_ONLY) {
  await this.verifyOrderReducesPosition(
    {
      productId: product.id,
      subaccountId: subaccount.id,
      ticker: product.ticker,
      side,
      quantity: dto.data.quantity,
    },
    tx
  );
}
```

Consider the following scenario:

- (1) Trader posts a normal limit order (increase size) while the product is OPEN.
- (2) Order sits on the Nekuti order book waiting for a match.
- (3) Product is updated to `CLOSE_ONLY`.
- (4) Indexer updates `product.status = CLOSE_ONLY` in the Sequencer DB.
- (5) The dormant limit order finally matches and the match is relayed on-chain
- (6) `_verifyNotReduceOnlyOrProductCloseOnly` sees an increase order against a `CLOSE_ONLY` product and reverts, aborting the entire batch.

H-04 Order Match DoS With Close Only Product

CONTINUED

Ultimately, it is possible to DoS all processing of a batch when a product is switched to CLOSE_ONLY.

— RECOMMENDATION

Consider working with Nekuti's engine to prevent increase orders from being executed upon CLOSE_ONLY mode.

— RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/sequencer/commit/ea099dc85ab02670503816f3d8aff3845eb85dc8>

HIGH **H-05**

ACKNOWLEDGED

Silent Precision-Loss in Mark-Price Publishing Path

CATEGORY Logical Error
LOCATION price-publisher.worker.ts
REVIEW Main Review

DESCRIPTION

The mark-price obtained from Pyth is stored internally as an 18-decimal `bigint`. Immediately before it is sent to the engine, that integer is coerced into a JavaScript number (IEEE-754 double) and divided by $1 \text{e}18$:

```
const productsWithPrice = productFilteredPrices.map(({ product, price }) => ({
  product,
  price: Gwei.toDecimal(price), // <---- CONVERSION TO NUMBER (only 53 bits precision)
}));
const productSnapshotData = productFilteredPrices.map(
  ({ product, markPrice, price }) =>
  ({
    productId: product.id.toString(),
    price: Gwei.fmt(Gwei.toGwei(price)),
    exponent: markPrice.exponent,
    onchainData: markPrice.onchainData,
    timestamp: markPrice.timestamp,
  }) satisfies PriceSnapshotData['prices'][number]
);
```

A double carries only 53 bits of integer precision, about 15–17 decimal significant digits. For prices in the 10^4 – 10^5 range, the fractional resolution collapses to the 7th or 8th decimal place ($\approx 10 \mu\text{USD}$). This looks tiny, but when multiplied by large position sizes or used in funding calculations every funding interval it can bias payouts.

Because the cast happens on every publish cycle the sequencer and margin engine are continuously working with a rounded value that diverges from the exact oracle feed. Audit trails will never match the original Pyth data bit-for-bit.

Code walkthrough

1. Normalization from Pyth (18-dec `bigint`). The 18-dec integer is cached in Redis:

H-05 Silent Precision-Loss in Mark-Price Publishing Path

CONTINUED

```
// File: src/modules/oracle/oracle.service.ts - transformMessageToMarkPriceCache
const price = BigInt(priceFeed.price) * 10n ** BigInt(GWEI_BASE + priceFeed.exponent);
// exponent = -8 ⇒ multiply by 10^10 ⇒ 18-dec fixed-point bigint
```

1. Retrieval and float cast:

```
// File: src/modules/root/price-publisher/price-publisher.worker.ts - inside publish()
const productsWithPrice = productFilteredPrices.map(({ product, price }) => ({
  product,
  price: Gwei.toDecimal(price), // <---- precision loss here
}));
```

1. Transmission to engine:

```
// File: src/modules/engine/engine.service.ts - updateInstrumentMarkPrice(...)
async updateInstrumentMarkPrice(
  priceSnapshotId: bigint,
  marks: { product: Selectable<Product>; price: number }[],
  options?: AxiosRequestConfig
) {
  const instrumentPrices = marks.map(
    ({ product, price }) =>
    ({
      instrument: this.getInstrumentName(product),
      // Engine calculates the "first bankruptcy price", marks are capped at this threshold,
      // prevents negative equity
      // scenarios, liquidation proceeds at the "first bankruptcy price".
      //
      // @see: https://nekuti.com/docs/maintenance_margin/markpricecap/
      //
      // This also acts as a protection mechanism if our oracle provider supplies a bad price. It
      // would simply cap it
      // at the first bankruptcy to avoid cascading liquidations.
      markPriceCapBehavior: 'CapToFirstBankruptcyPrice',
      price,
    }) as const
  );
  try {
    const updateId = this.getMarkUpdateIdFromPriceSnapshotId(priceSnapshotId);
    await this.sdk.instrumentUpdateMarkPrices(
      { instrumentPrices, updateId },
```

H-05 Silent Precision-Loss in Mark-Price Publishing Path

CONTINUED

```
options ? { ...this.requestOptions, ...options } : this.requestOptions
);
} catch (err) {
  this.handleUnexpectedRequestFailure(err, 'Update mark prices failed', getLogContext({
    priceSnapshotId }));
}
}
```

Once the value leaves `price-publisher.worker.ts` it is already rounded, every downstream consumer works with the degraded float.

— RECOMMENDATION

Consider retaining full-precision fixed-point until the very last serialisation step.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Insecure WebSocket Connection

CATEGORY Connections
LOCATION engine.service.ts:1016
REVIEW Main Review

DESCRIPTION

An unencrypted WebSocket `ws://` connection is established by the Meridian Sequencer Engine Listener, allowing an attacker on the same network as the server or user to intercept WebSocket communication data in plaintext via a Man-in-the-Middle attack.

RECOMMENDATION

Establish a WebSocket connection over an encrypted TLS connection using `wss://`.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

HTTP And WebSocket Requests Resolve To The Load-balancer IP As trustProxy Is false

CATEGORY DoS
LOCATION rate-limit.service.ts
REVIEW Main Review

DESCRIPTION

The sequencer implements two parallel rate-limit entry points:

- Transport | Key generation path | Relevant code
- HTTP (REST) | request.ip | RateLimitService.getIdentifierHttp()
- WebSocket (Socket.IO) | resolveClientAddress() | RateLimitGuard.wsResponseHandler()

HTTP flow:

```
// src/modules/rate-limit/rate-limit.service.ts
getIdentifierHttp(req: FastifyRequest) {
  return request.ip; // ← raw peer address
}
```

`request.ip` is rewritten to the first element of X-Forwarded-For only if Fastify is started with `trustProxy: true`, which is not the case as:

```
// src/modules/config/server-config.ts
import dotenv from 'dotenv';
import { Logger } from '@nestjs/common';
import { z } from 'zod';
import { TrustProxySchema } from '../../shared/schema';
import { getDotenvFilename, getEnvironment } from './env.provider';

const DEFAULT_CONFIG = {
  trustProxy: 'false',
} as const;

// This is a workaround to allow Fastify to be configured before the DI container is built
const ServerConfigSchema = z.object({
  trustProxy: TrustProxySchema.default(DEFAULT_CONFIG.trustProxy),
});

export type ServerConfig = z.infer<typeof ServerConfigSchema>;
```

M-02 HTTP And WebSocket Requests Resolve To The Load-balancer IP As trustProxy Is false

CONTINUED

```
export const getServerConfig = () => {
  const logger = new Logger('ServerConfig');
  const nodeEnv = getEnvironment();
  const path = getDotenvFilename(nodeEnv);
  if (path) {
    dotenv.config({ path });
  }
  const { data, error } = ServerConfigSchema.safeParse({
    trustProxy: process.env.HTTP_TRUST_PROXY,
  });
  if (error) {
    logger.error('Failed server config validation');
    throw error;
  }
  return data;
};
```

Therefore, (client → LB → Sequencer) every request appears to originate from the proxy and all users share a single Redis bucket: `<prefix>:ip:general:203.0.113.7`.

WebSocket flow

```
// src/shared/ws/ws-connection.ts
export const resolveClientAddress = (handshake, trustProxy) => {
  if (!trustProxy) {
    return handshake.address; // ← raw TCP peer (proxy IP)
  }
  /* ... parse X-Forwarded-For when trustProxy > 0 ... */
}
```

Because the configuration file sets `sequencer.ws.trustProxy = false` by default, the helper also returns the proxy's address (`handshake.address = socket.remoteAddress`):

```
// src/modules/config/dto/sequencer.dto.ts
const DEFAULT_CONFIG = {
  logLevel: 'info',
  logToFile: 'false',
  http: {
    port: 3000,
    // NOTE: 'trustProxy' see 'server-config.ts'.
  },
  ws: {
```

M-02 HTTP And WebSocket Requests Resolve To The Load-balancer IP As trustProxy Is false

CONTINUED

```
port: 3001,  
// @see: https://fastify.dev/docs/latest/Reference/Server/#trustproxy  
trustProxy: 'false',  
}
```

Only when operators manually set `trustProxy` to `true` the code looks at the rightmost entries of `X-Forwarded-For` and recover the real client IP.

Resulting behaviour

- Deployment | trustProxy | HTTP key | WS key | Effect
- Direct (no LB) | false | real IP | real IP | Works correctly.
- Behind LB / CDN | false | proxy IP | proxy IP | All users throttled together; attacker can DoS legitimate traffic.
- Behind LB / CDN | HTTP: true, WS: false | real IP | proxy IP | Inconsistent limits, WS remains vulnerable.
- Behind LB / CDN | true (both) | real IP | real IP | Correct / expected.

With the current setup any moderately active user exhausts the shared bucket and triggers `429 Too Many Requests` for every other client, both over REST and WebSocket APIs. On the other hand, rate-limit headers (`X-RateLimit-*`) and metrics misrepresent the real distribution of traffic.

— RECOMMENDATION

Consider setting `trustProxy` flag to true following the Fastify documentation:
<https://fastify.dev/docs/latest/Reference/Server/#trustproxy>

On the other hand, unit-test both layers: send requests with `X-Forwarded-For: <client>,<proxy>`, assert that HTTP and WS produce the same key.

Implementing the above ensures that every real end-user is mapped to its own Redis bucket regardless of the presence of reverse proxies.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

Malicious Subaccount Can Prevent Actions

CATEGORY Logical Error
LOCATION order.service.ts
REVIEW Main Review

DESCRIPTION

When the subaccount performs an action off-chain, the rate limit is consumed on the parent account. For example in function `cancel`: `await this.rateLimitAccountService.consume(subaccount.account, RateLimitPoints.LOW);`

This allows a singular malicious subaccount to drain the remaining rate limit points for the entire account, consequently preventing other subaccounts from cancelling or submitting orders which can ultimately lead to losing trades/failed hedges/fund loss.

RECOMMENDATION

Consider enforcing a sub-account specific ratelimit as well.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

RESOLVED

Mixed-case Ethereum Addresses Bypass Per-Account Rate-Limit Buckets

CATEGORY DoS
LOCATION `rate-limit-account.service.ts`
REVIEW Main Review

— DESCRIPTION

The account-level limiter uses the wallet address string as its Redis key:

M-04 Mixed-case Ethereum Addresses Bypass Per-Account Rate-Limit Buckets **CONTINUED**

```
// src/modules/rate-limit/rate-limit-account.service.ts
async consume(account: Address, pointsConsumed: number) {
  const rateLimiter = this.rateLimitService.getRateLimiter(RateLimitType.ACCOUNT);
  try {
    const [result] = await Promise.all([
      rateLimiter.general.consume(account, pointsConsumed),
      rateLimiter.burst.consume(account, pointsConsumed),
    ]);
    if ([Environment.TEST, Environment.CI].includes(this.configService.env)) {
      return;
    }
    // This intercepts the underlying server response and not the fastifyReply type
    const res = this.cls.get(CLS_RES) as ServerResponse | undefined;
    if (!res) {
      throw new Error('No response object found in cls');
    }
    res.setHeader('X-RateLimit-Account-Retry-After', Math.ceil(result.msBeforeNext / 1000).toString());
    res.setHeader('X-RateLimit-Account-Limit', (result.consumedPoints + result.remainingPoints).toString());
    res.setHeader('X-RateLimit-Account-Remaining', result.remainingPoints.toString());
  } catch (err) {
    if (err instanceof RateLimiterRes) {
      throw new RateLimitLimitError({ retryAfter: Math.ceil(err.msBeforeNext / 1000) });
    }
    throw err;
  }
}
```

Because Ethereum addresses are case-insensitive, multiple textual variants represent the same on-chain identity. Example variants (all the same wallet):

```
0xF3ca3b7127A9E546f69E3C47368Fb51d8F405F40
0xf3CA3B7127a9e546F69e3C47368fB51d8F405f40
0xf3ca3b7127a9e546f69e3c47368fb51d8f405f40 // all-lower
```

Each variant becomes a separate Redis key (<prefix>:account:*<address>) so the attacker's true call rate equals the sum of all buckets they create. Therefore, a single user can exceed quota by a factor equal to the number of mixed-case permutations they generate (millions).

M-04 Mixed-case Ethereum Addresses Bypass Per-Account Rate-Limit Buckets **CONTINUED****RECOMMENDATION**

Normalise addresses once and consistently:

```
import { getAddress } from 'viem'; // ensures EIP-55 checksum
function normalise(addr: Address): string {
  // Option A: force lower-case
  // return getAddress(addr).toLowerCase();
  // Option B (cleaner): use canonical EIP-55 checksum
  return getAddress(addr); // always same casing for same address
}
```

Apply the helper function when generating the key:

```
const key = normalise(account);
await Promise.all([
  rl.general.consume(key, points),
  rl.burst.consume(key, points),
]);
```

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/sequencer/commit/1505b7667f769181b3ac677864d019322fc39230>

MEDIUM **M-06**

RESOLVED

Clients Can Never Unsubscribe From Liquidation WebSocket Stream

CATEGORY Logical Error
LOCATION stream.service.ts
REVIEW Main Review

DESCRIPTION

`StreamService` implements symmetrical helpers to join/leave Socket.IO rooms per stream type. While the subscribe path for liquidations is correct, the unsubscribe path contains a copy-paste error:

```
// src/modules/stream/stream.service.ts
async unsubscribeLiquidation(subaccountExtId: string, socket: Socket) {
  // ✗ uses Order-Update helper instead of Liquidation helper
  await this.updateOrderUpdateSubscription(subaccountExtId, socket, false);
}
```

- The call should be `updateLiquidationSubscription`.
- Because the wrong helper computes a different room key (`ORDER_UPDATE:<id>`), the `socket.leave()` call is executed on a room the client was never in, leaving the `SUBACCOUNT_LIQUIDATION:<id>` room untouched.
- The socket therefore continues to receive liquidation events after the front-end believes it has unsubscribed.

Because of this, traders may inadvertently receive liquidation data for accounts they no longer track. Dangling room membership keeps extra entries in Redis adapter sets; at scale this increases memory, key-scans, and broadcast fan-out, degrading latency.

RECOMMENDATION

Fix the helper reference

```
async unsubscribeLiquidation(subaccountExtId: string, socket: Socket) {
  await this.updateLiquidationSubscription(subaccountExtId, socket, false); // ✓ correct
  helper
}
```

M-06 Clients Can Never Unsubscribe From Liquidation WebSocket StreamCONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/sequencer/pull/714>

MEDIUM **M-07**

ACKNOWLEDGED

Precision-loss In getProductPriceRange

CATEGORY	Logical Error
LOCATION	product.service.ts
REVIEW	Main Review

DESCRIPTION

The `getProductPriceRange(initialPrice, tickSize, ...)` function is responsible for defining the minimum and maximum allowed prices for every product:

```
private getProductPriceRange(  
  initialPrice: number,  
  tickSize: number,  
  levelsByDirection = MAX_PRICE_LEVELS_BY_DIRECTION,  
  maxPriceLevels = MAX_PRICE_LEVELS  
) {  
  const levelsByPriceTicks = levelsByDirection * tickSize;  
  const minPrice = Math.max(0, Math.round(initialPrice - levelsByPriceTicks));  
  const maxPrice = Math.round(initialPrice + levelsByPriceTicks);  
  // An instrument can have a maximum of 2,147,483,647 price levels.  
  if ((maxPrice - minPrice) / tickSize > maxPriceLevels) {  
    const logContext = getLogContext({ minPrice, maxPrice, maxPriceLevels, levelsByDirection });  
    throw new ProductError('Price range exceeds maxPriceLevels ${logContext}');  
  }  
  return { minPrice, maxPrice };  
}
```

- All arithmetic is performed with JavaScript number (IEEE-754 binary64).
- `initialPrice` is a plain number parsed from the off-chain config.
- `tickSize` is converted from on-chain gwei to a number via `Gwei.toDecimal`.

Because number has only 53 bits of integer precision:

- Any value $> |2^{53}|$ can not be represented precisely.
- Small decimal fractions (e.g. 0.000000001) cannot be represented at all.

Consequences

M-07 Precision-loss In getProductPriceRange

CONTINUED

1. `levelsByPriceTicks` under- or over-flows after rounding, so `minPrice` can be equal to `maxPrice` or `minPrice` can actually become negative.
2. The guard that enforces the 2 147483 647 level cap can pass even though the true number of ticks is larger.
3. Orders outside the rounded range are rejected by the matching engine while orders inside may be mismargined and could lead to undercollateralization or insolvency.

Concrete examples

- Scenario | Input | Observed result
- Large notional / tiny tick | `initialPrice` = 1 200 000 000 | `tickSize` = 1 e-9 | `levelsByPriceTicks` underflows to 0.01 → `Math.round` erases it → `minPrice == maxPrice`, collapsing the order-book width to 1 tick.
- Value just above 2^{53} | `initialPrice` = 9 007 199 254 740 993 ($2^{53} + 1$) | JS stores it as 2^{53} , book range is computed with a wrong price

— RECOMMENDATION

Migrate the entire helper to fixed-point or bigint arithmetic. A minimal, backwards-compatible patch:

```
const SCALE = 1_000_000_000n; // gwei
const toInt = (x: string|number) => BigInt(Math.round(Number(x) * 1e9));
const initial = toInt(initialPrice);
const tick = toInt(tickSize);
const span = BigInt(levelsByDirection) * tick;
const min = initial > span ? initial - span : 0n;
const max = initial + span;
if ((max - min) / tick > BigInt(MAX_PRICE_LEVELS))
  throw new ProductError('Price range exceeds limit');
return {
  minPrice: (min / SCALE).toString(),
  maxPrice: (max / SCALE).toString(),
};
```

Additional hardening:

1. Fail fast if either `initialPrice` or `tickSize` is not a safe integer or if `tickSize < 1e6` USD.
2. Lint rule or type alias (type `USD = bigint`) forbidding raw number for monetary units project-wide.

M-07 Precision-loss In getProductPriceRange

CONTINUED

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-09**

RESOLVED

linkSignerExpiryDays Default Value Mismatch

CATEGORY Warning
LOCATION sequencer.dto.ts
REVIEW Main Review

DESCRIPTION

In the `sequencer.dto.ts` file the `DEFAULT_CONFIG` is defined as:

```
const DEFAULT_CONFIG = {  
  ...  
  account: {  
    linkSignerLimitInPeriod: 5,  
    linkSignerLimitPeriodDays: 7, // ❶ correct: 7 days  
    linkSignerExpiryDays: 30, // ❷ correct: 30 days (intended default)  
    maxPositions: 50,  
  },  
  ...  
} as const;
```

On the other hand, the Zod schema for the account is defined as:

```
export const SequencerConfig = z.object({  
  ...  
  account: z.object({  
    linkSignerLimitInPeriod: z.coerce.number().positive().int()  
      .default(DEFAULT_CONFIG.account.linkSignerLimitInPeriod),  
    linkSignerLimitPeriodDays: z.coerce.number().positive().int()  
      .default(DEFAULT_CONFIG.account.linkSignerLimitPeriodDays),  
    // 🚩 BUG: default points to the *wrong* constant  
    linkSignerExpiryDays: z.coerce.number().positive().int()  
      .default(DEFAULT_CONFIG.account.linkSignerLimitPeriodDays), // should be linkSignerExpiryDays  
    maxPositions: z.coerce.number().positive().int().min(1)  
      .default(DEFAULT_CONFIG.account.maxPositions),  
  }),  
  ...  
});
```

M-09 linkSignerExpiryDays Default Value Mismatch

CONTINUED

Problem line:

```
.default(DEFAULT_CONFIG.account.linkSignerLimitPeriodDays)
```

Instead of:

```
.default(DEFAULT_CONFIG.account.linkSignerExpiryDays)
```

Because of this copy-&-paste error, the default `linkSignerExpiryDays` becomes 7 instead of the intended 30. Therefore, after a week, traders that rely on a linked hot-wallet suddenly get `SIGNER_EXPIRED` errors when submitting orders or cancelling positions.

— RECOMMENDATION

Update the `sequencer.dto.ts` file as shown below:

```
- linkSignerExpiryDays:
z.coerce.number().positive().int().default(DEFAULT_CONFIG.account.linkSignerLimitPeriodDays),
+ linkSignerExpiryDays:
z.coerce.number().positive().int().default(DEFAULT_CONFIG.account.linkSignerExpiryDays),
```

— RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/sequencer/commit/740a1c58bc20c4f4f1987c44f45ec8747b4f4614>

MEDIUM **M-10**

ACKNOWLEDGED

safe-array-where-in Plugin Returns Wrong Results For NOT IN ()

CATEGORY Logical Error
LOCATION kysely-safe-array-where-in.ext.ts
REVIEW Main Review

DESCRIPTION

`WithSafeArrayWhereInPlugin` is a custom Kysely plugin that tries to protect developers from the Postgres syntax error `"syntax error at or near ' ) '"` that happens when an `IN` list is empty. It rewrites every run-time empty list to `IN (NULL)`.

```
if (this.#isInOperatorNode(operator) && PrimitiveValueListNode.is(rightOperand)) {  
  if (Array.isArray(values) && values.length === 0) {  
    return {  
      ...node,  
      rightOperand: {  
        ...rightOperand,  
        // 🐛 injects a single NULL  
        values: [null],  
      },  
    };  
  }  
}
```

For positive tests (`... WHERE id IN ( )`) the replacement is harmless: `IN (NULL)` evaluates to `FALSE`, yielding the expected empty result set. For negative tests (`... WHERE id NOT IN ( )`) the same rewrite yields `NOT IN (NULL)`.

In Postgres the expression `NOT IN (NULL)` returns `NULL`, which the `WHERE` clause treats as `FALSE` and therefore all rows are filtered out. Instead of returning every row, the query returns none, breaking business logic or causing subtle data loss in delete/update statements.

Because the plugin is injected globally, any repository call that uses array filters and passes an empty array may be affected without the developer noticing.

M-10 safe-array-where-in Plugin Returns Wrong Results For NOT IN ()

CONTINUED

RECOMMENDATION

Refine the transformer so that it emits a boolean literal when the list is empty:

- ... IN () → FALSE
- ... NOT IN () → TRUE

This preserves SQL's three-valued logic and avoids the NULL trap. For example:

```
class WithSafeArrayWhereInTransformer extends OperationNodeTransformer {
  #isIn(node: OperatorNode) {
    return node.operator === 'in';
  }
  #isNotIn(node: OperatorNode) {
    return node.operator === 'not in';
  }
  protected override transformBinaryOperation(node: BinaryOperationNode): BinaryOperationNode {
    const { operator, rightOperand } = node;
    if (PrimitiveValueListNode.is(rightOperand) && rightOperand.values.length === 0) {
      if (OperatorNode.is(operator)) {
        if (this.#isIn(operator)) {
          // x IN () -> FALSE
          return sql'FALSE'.toOperationNode() as BinaryOperationNode;
        }
        if (this.#isNotIn(operator)) {
          // x NOT IN () -> TRUE
          return sql'TRUE'.toOperationNode() as BinaryOperationNode;
        }
      }
    }
    return node;
  }
}
```

RESOLUTION

Status: Acknowledged.

MEDIUM **M-11**

ACKNOWLEDGED

Precision Loss On Funding

CATEGORY Precision
LOCATION funding.service.ts
REVIEW Main Review

DESCRIPTION

Because the Nekuti engine utilizes IEEE 754 whereas the Sequencer (and contracts) use BigInt, the charged funding can vary 1 gwei between the systems. Although this is not a large discrepancy, this would cause a desync between the two systems that will grow with every funding charge.

For example:

- (1) Within `updateFunding`, the `chargePerUnitUsd = 9007199254740993n` -- a BigInt
- (2) Nekuti engine processes the charge as a double such that `chargePerUnitUsd = 9007199254740992` -- IEEE754 format
- (3) `handleFundingCharge` reads the stored charge in the DB which is `9007199254740993n` and not the `9007199254740992` charged on the engine.
- (4) `UpdatingFunding` action is sent on-chain with `9007199254740993n`
- (5) Ultimately there is a desync between engine and Sequencer/engine with the funding charged, and gap can widen over time.

RECOMMENDATION

Consider if Nekuti engine can utilize the same precision or consider accounting for the discrepancy with a separate funding charge.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-12**

RESOLVED

Rate-limiting Imbalance On POST /order/cancel

CATEGORY Warning
LOCATION order.controller.ts
REVIEW Main Review

DESCRIPTION

The cancel endpoint is annotated with `@LowRateLimit()` in `order.controller.ts`, which translates to a single `RateLimitPoints.LOW` debit inside the service layer:

```
@Post('cancel')
@HttpCode(HttpStatus.ACCEPTED)
@LowRateLimit() // <-- low-cost bucket
async cancel(@Body() dto: CancelOrderDto) {
  await this.whitelistService.validateAddress(dto.data.sender);
  return toListOfCancelOrderResultDtos(await this.orderService.cancel(dto));
}
```

At the same time `cancel-order.dto.ts` permits an array of up to 256 UUIDs through the `MAX_ORDER_CANCEL_BATCH_SIZE` constant:

```
const MAX_ORDER_CANCEL_BATCH_SIZE = 256;
@IsUUID('4', { each: true })
@ArrayMinSize(1)
@ArrayMaxSize(MAX_ORDER_CANCEL_BATCH_SIZE) // <-- 256 ids
readonly orderIds!: string[];
```

During execution, `OrderService.cancel` pipes the entire list straight to the matching engine with a single call:

```
const results = await this.engineService.cancel({ subaccount, orderIds: orderIdsToCancel });
```

From the engine's perspective this represents 256 individual cancellation operations, each triggers order-book mutation, audit entry creation, nonce update and event emission. Yet the API gateway and rate-limit accounting treat the whole bundle as one cheap request. An attacker can therefore submit cancel requests in a tight loop and multiply their effective load on the engine by a factor of 256 relative to submit requests, which are billed at

M-12 Rate-limiting Imbalance On POST /order/cancel

CONTINUED

`@MediumRateLimit()` and only handle one order at a time. This asymmetry creates an avenue for excessive resource consumption or denial-of-service against the core matching engine.

— RECOMMENDATION

Re-align the cost of a cancellation request with the work it imposes. The simplest fix is to debit rate-limit points proportionally to `orderIds.length`, for example:

```
// inside OrderService.cancel
await this.rateLimitAccountService.consume(
  subaccount.account,
  dto.data.orderIds.length * RateLimitPoints.LOW
);
```

Alternatively lower `MAX_ORDER_CANCEL_BATCH_SIZE` to a more conservative value (e.g. 32) or introduce a dedicated `HIGH` tier for large batches. Whichever strategy is chosen, ensure the rate-limit schedule for cancellation cannot be abused to produce significantly more engine work per token than order submission or modification calls.

— RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/sequencer/commit/4180ca070bd57f21d437e05a1a1aff7ab54786bc>

MEDIUM **M-13**

RESOLVED

Revoked Signer Submits Order With Race Condition

CATEGORY Logical Error
LOCATION Global
REVIEW Main Review

DESCRIPTION

Concurrent `submit` and `revokeSigner` calls can interleave under the default READ COMMITTED isolation. Consequently, the following race condition is possible with two concurrent requests:

- Order submission flow starts and signer appears active, hence `verifySubaccountAllowSigners` validation passes successfully.
- Order flow stalls during async validations (Promise.all, DB checks, etc.).
- Revoke flow sees no pending orders yet, deactivates signer, finishes transaction.
- Order flow continues and inserts the new order

Ultimately, the system now has an order that will be matched for a revoked signer. Because `revokeSigner` on-chain excludes the account, once the match is relayed on-chain it will revert with `ExcludedAccount` and the entire batch of actions will be DoS'd

RECOMMENDATION

Consider enforcing a lock so each flow cannot read stale data until the other flow commits or rolls back.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/sequencer/pull/767>

MEDIUM **M-14**

ACKNOWLEDGED

Incorrect Cache Refresh Logic in `getWithdrawLockout`

CATEGORY Logical Error
LOCATION `subaccount.service.ts`
REVIEW Main Review

DESCRIPTION

The `getWithdrawLockout` method in the `SubaccountService` class suffers from a logical flaw in its cache-refresh condition, causing the system to rely on outdated withdrawal lockout periods. This method is designed to cache the lockout value, which is retrieved from the exchange contract and refresh it periodically based on a time-to-live (TTL) threshold. However, due to an error in the condition that determines when the cache should be updated, the method refreshes the cache when it is still fresh and fails to do so when it becomes stale. This leads to the system using incorrect lockout values, which in turn affects the accuracy of withdrawal readiness indicators like `isReady` and `readyAt`. Therefore, users may encounter failed withdrawal attempts or miss opportunities to access their funds, undermining the platform's reliability.

The issue stems from the cache-refresh condition in the current implementation. Consider the following code snippet:

```
if (  
  !this.cachedLockout ||  
  !this.lastCachedLockoutAt ||  
  this.lastCachedLockoutAt > Date.now() - this.LOCKOUT_CACHE_TTL_MS  
) {  
  // Refresh logic  
}
```

In this logic, the third part of the condition `(this.lastCachedLockoutAt > Date.now() - this.LOCKOUT_CACHE_TTL_MS)` is where the error lies. This check determines whether the last refresh time is more recent than the staleness threshold, calculated as the current time minus the TTL. However, this is the opposite of the intended behavior. When the cache is fresh, for example, if it was refreshed 1 minute ago and the TTL is 5 minutes, the condition evaluates to `true` because `this.lastCachedLockoutAt` is greater than `Date.now() - this.LOCKOUT_CACHE_TTL_MS`. This triggers an unnecessary refresh. Conversely, when the cache is stale, for instance, if it was refreshed 10 minutes ago with a 5-minute TTL, the

M-14 Incorrect Cache Refresh Logic in getWithdrawLockout

CONTINUED

condition evaluates to `false`, and the cache is not refreshed, leaving the system to operate with outdated data.

RECOMMENDATION

To address this issue, the condition must be updated to correctly identify when the cache is stale and needs updating. The condition should trigger a refresh if the cache is missing, if the last refresh time is untracked, or if the last refresh occurred before the staleness threshold. Here's the corrected implementation:

```
private async getWithdrawLockout() {
  if (
    !this.cachedLockout ||
    !this.lastCachedLockoutAt ||
    this.lastCachedLockoutAt < Date.now() - this.LOCKOUT_CACHE_TTL_MS
  ) {
    const lockout = await this.rpcService.exchange.read.getExchangeWithdrawLockout();
    this.cachedLockout = lockout;
    this.lastCachedLockoutAt = Date.now();
  }
  return this.cachedLockout;
}
```

In this corrected version, the third part of the condition (`this.lastCachedLockoutAt < Date.now() - this.LOCKOUT_CACHE_TTL_MS`) now checks whether the last refresh time is older than the staleness threshold. This ensures that the cache is refreshed only when necessary: when it is missing, untracked, or stale.

RESOLUTION

Status: Acknowledged.

<https://github.com/meridianxyz/contracts/pull/174>

MEDIUM **M-16**

RESOLVED

Incorrect NULL Comparison in isLiquidated Filter

CATEGORY Logical Error
LOCATION position.repository.ts
REVIEW Main Review

DESCRIPTION

In `position.repository.ts` the liquidation filter is implemented as:

```
if (typeof filters.isLiquidated === 'boolean') {  
  q = q.where('liquidationId', '!=', null);  
}
```

In PostgreSQL (and in most SQL dialects) the comparison `!= NULL` is never `true` as it evaluates to `NULL`, which behaves as unknown in a `WHERE` clause and therefore filters nothing. When `isLiquidated` is `true` the query unintentionally returns zero rows, and when it is `false` the surrounding condition mistakenly applies the same expression, again returning an empty set. Users or downstream services relying on this endpoint will be unable to retrieve liquidated positions, leading to missing risk metrics, misleading dashboards and possible reconciliation failures.

RECOMMENDATION

Replace the expression with an explicit `IS [NOT] NULL` check that branches on the requested boolean:

```
if (filters.isLiquidated === true) {  
  q = q.where('liquidationId', 'is not', null); // only liquidated rows  
} else if (filters.isLiquidated === false) {  
  q = q.where('liquidationId', 'is', null); // only non-liquidated rows  
}
```

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/sequencer/pull/763>

MEDIUM **M-17**

ACKNOWLEDGED

Signer Revoke Prevented With Unfillable Order

CATEGORY DoS
LOCATION linked-signer.service.ts
REVIEW Main Review

— DESCRIPTION

A pre-condition for a successful `revokeSigner` call on the API endpoint is that the subaccount does not have any orders that are pending fills, which is checked via function `verifyNoFillableOrders`.

This allows for a potential DoS opportunity where the linked signer creates an unfillable order e.g. extreme limit price. Because the order is pending and `verifyNoFillableOrders` will throw, the account sender will not be able to revoke the linked signer.

— RECOMMENDATION

Clearly document this risk.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-18**

ACKNOWLEDGED

Engine Fill Cost Precision Loss

CATEGORY	Logical Error
LOCATION	Liquidation.sol: 186, Sequencer
REVIEW	Main Review

DESCRIPTION

Due to IEEE754 precision from the Nekuti matching engine, the quantity liquidated multiplied by the fill price will not always match the cost of the position, as the cost may be 1 wei larger.

This will ultimately lead to a desync with on-chain liquidation, since the cost is re-calculated with BigInt precision, rounding down: `int128 markCost =`

```
(rt.price.mulDecimal(Math.abs(action.products[i].sizeDelta))).toInt128();
```

Because the `markCost` may be smaller 1 wei for each product, in the case of short positions liquidated the cumulative PnL would increase and `LiquidateAboveMaintenanceMargin` revert may be triggered even though the liquidation was valid from the Engine's perspective. This will also DoS all other actions that are being processed in the batch.

Note that there will also be discrepancy within `_settleLiquidatorPosition` when calculating the `uint256 closeCost = (cost * Math.abs(oldSize)) / Math.abs(sizeDelta);` since `cost` which is from the Nekuti engine may be 1 wei larger due to the IEEE754 precision.

RECOMMENDATION

The best way to solve this is to have Nekuti use the exact same precision as the Sequencer and on-chain. In general, rounding should be in favor of the protocol rather than the user. Any discrepancy for the Liquidator can be manually accounted for.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-19**

RESOLVED

Engine And Sequencer Discrepancy For Unfilled Market Order

CATEGORY Logical Error
LOCATION Global
REVIEW Main Review

— DESCRIPTION

When a market order is cancelled/rejected upon submission to Nekuti, the order's status within the Sequencer is `SUBMITTED`, but from the EngineService the order's status is `CANCELED`. Consequently, the order repository in the Sequencer contains stale data for the order.

— RECOMMENDATION

Consider reading the `/submit` response and adjusting the order repository based on `ordStatus`, or having Nekuti send a cancel execution group if the Market order is outright cancelled.

PROOF OF CONCEPT <https://github.com/GuardianOrg/meridian-sequencer/pull/2>

— RESOLUTION

Status: Resolved.

LOW **L-01**

RESOLVED

Typo in capActionsUsingEstimatedGasUsage

CATEGORY	Logical Error
LOCATION	relayer.service.ts
REVIEW	Main Review

— DESCRIPTION

In the `RelayerService` class, the `capActionsUsingEstimatedGasUsage` method contains a typo in one of its return statements. Specifically, the second return statement uses `gsEstimate` instead of `gasEstimate`:

```
return { gsEstimate: sumGasEstimate, cappedActions: actions };
```

This should be:

```
return { gasEstimate: sumGasEstimate, cappedActions: actions };
```

The method is expected to return an object with properties `gasEstimate` and `cappedActions`. Due to the typo, when this return path is executed (i.e., when all actions fit within the `gasLimit`), the returned object has a key named `gsEstimate` instead of `gasEstimate`. This inconsistency causes issues in the calling method `getActionsAndSimulate`, where the object is destructured as:

```
const { gasEstimate, cappedActions } = this.capActionsUsingEstimatedGasUsage(actionsBatch, gasLimit);
```

Because `gsEstimate` is returned instead of `gasEstimate`, the `gasEstimate` variable becomes undefined, leading to potential runtime errors or incorrect behavior in subsequent code that relies on this value such as the logger.

— RECOMMENDATION

Correct the typo in the `capActionsUsingEstimatedGasUsage` function.

— RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/sequencer/commit/465a3134bd29237cad2c049c510a04710cc9984a>

LOW **L-01.2**

RESOLVED

Insufficient Checks In depositOnBehalf

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/contractsmeridian-sequencer1-team2/blob/0c158ff08758dc60df2d2d63c2907edd6fe7fe58/src/CollateralManager.sol#L289
REVIEW	Main Review

DESCRIPTION

The depositor can make a deposit on behalf of another account and its subaccount, but only if it is the first deposit or if the subaccount has already been registered.

```
if (!account.hasRegistered[subaccount] && account.registeredSubaccounts != 0) {  
    revert DepositOnBehalfNotAllowed(msg.sender, receiver, subaccount);  
}
```

However, this check is insufficient because the `account.registeredSubaccounts` count is incremented only after the deposit is finalized. As a result, the depositor's action is incorrectly treated as the first deposit, even when the actual first deposit for the account is still pending.

Due to this behavior the depositor can register multiple, different subaccounts for the same account through multiple `depositOnBehalf` calls within a single transaction. The check will not revert, as each of these deposits is treated as the first deposit for the account. Consequently, all subaccounts will be registered when the deposits are finalized and the comment "avoiding unnecessary subaccounts from being created" is violated.

RECOMMENDATION

Check whether there is a pending deposit for the account before allowing a depositor to deposit on its behalf. Alternatively, disallow depositors from making a first deposit on behalf of an account, and only permit deposits if the subaccount has already been registered.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/commit/d808654784a3312ff4fc02d8bf0e9ad14456c004>

LOW **L-02**

ACKNOWLEDGED

Possible Inaccuracy In Funding Rate Calculation Due to Clamping Mechanism

CATEGORY Warning
LOCATION funding.service.ts
REVIEW Main Review

DESCRIPTION

The current implementation of the funding rate calculation in the `FundingService` class employs a clamping mechanism that sets the funding rate to zero when the absolute value of the average basis (the difference between the mid price and the mark price) is less than or equal to a predefined clamp rate, currently set at 0.01%. This mechanism is designed to filter out noise and prevent overcorrections in the funding rate. However, it introduces significant risks that could result in unfair trading conditions and potential financial losses for traders.

The clamping logic operates as follows: if the absolute value of the average basis is less than or equal to the clamp rate, the funding rate is set to zero before any baseline rate is added. If the basis exceeds the clamp rate, the full basis is used in the funding rate calculation, adjusted by the baseline rate, and then capped within a maximum range. The relevant code in the `boundFundingRate` method is shown below:

```
const clampedRate = Gwei.abs(initialRate) <= clampRate ? 0n : initialRate;  
const fundingRate = clampedRate + baselineRate;  
const cappedRate = fundingRate > 0n ? Gwei.min(fundingRate, maxRate) : Gwei.max(fundingRate, -maxRate);
```

This approach poses problems in scenarios where small but persistent price differences occur. For instance, if the basis remains consistently at 0.005%, which is below the clamp rate of 0.01%, the funding rate will be set to zero. This allows the price deviation to persist without any corrective action, potentially enabling one side, such as long positions, to benefit disproportionately from the uncorrected drift. Traders on the opposing side, such as short positions, may perceive this as unfair, as the funding mechanism fails to address these small but consistent deviations.

RECOMMENDATION

L-02 Possible Inaccuracy In Funding Rate Calculation Due to Clamping Mechanism

CONTINUED

To mitigate the risks and improve the fairness of the funding rate calculation, a possible approach is to replace the hard clamping mechanism where the funding rate is set to zero with a scaled funding rate that applies even when the basis is below the clamp rate. For instance, the funding rate could be calculated as a fraction of the basis relative to the clamp rate, as shown in the following example:

```
const scaledRate = (initialRate / clampRate) * someFactor;
```

This method ensures that small deviations still contribute to the funding rate, albeit at a reduced magnitude, preventing persistent uncorrected drifts while maintaining the goal of filtering out noise. The funding rate would then be adjusted by the baseline rate and capped as usual.

— RESOLUTION

Status: Acknowledged.

LOW **L-02.2**

ACKNOWLEDGED

No Onchain maxOpenInterestUsd Validation

CATEGORY	Validation
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Each perpetual product stores a hard upper limit on aggregate exposure:

```
struct Data {  
    ...  
    uint128 maxOpenInterestUsd; // configuration parameter  
    uint128 openInterest; // live long OI (native units)  
    ...  
}
```

`maxOpenInterestUsd` is intended to restrict total notional risk in the product, however, no function that mutates `product.openInterest` ever checks this ceiling:

- Order matching: `PerpEngine._settlePositionFill` adjusts `product.openInterest` after every fill, but never verifies that the resulting value stays $\leq$ `maxOpenInterestUsd`.
- Liquidations/forced transfers: `Liquidation._settleLiquidatorPosition` also increases `product.openInterest` when a liquidator inherits a long side, again without comparing against the cap.

— RECOMMENDATION

Document that the primary validation for `maxOpenInterestUsd` and `maxPositionNotionalUsd` is offchain. Otherwise, insert a guard at smart contract level every time `openInterest` is about to be updated, although great care has to be taken with `maxOpenInterestUsd` updates since the latest value can take effect immediately on-chain but still not be updated in the Engine, leading to a DoS'd batch.

— RESOLUTION

Status: Acknowledged.

LOW **L-03.2**

ACKNOWLEDGED

Nonce Usage Drift

CATEGORY Warning
LOCATION PerpEngine.sol: 567
REVIEW Main Review

— DESCRIPTION

When a user submits an order the nonce is consumed and can never be reused off-chain. During on-chain order fill settlement, an order's nonce is only marked used once the order is filled. If an order is cancelled after a partial fill, then the nonce for that order will not be marked as used on-chain.

This causes some discrepancy between on-chain and off-chain nonce reads which may confuse integrators/traders, although this is not a security issue as ultimately all actions must go through the Sequencer anyway.

— RECOMMENDATION

Clearly document this behavior

— RESOLUTION

Status: Acknowledged.

LOW **L-04.2**

RESOLVED

Lacking DisableInitializers

CATEGORY	Warning
LOCATION	ExchangeGateway.sol
REVIEW	Main Review

DESCRIPTION

The `ExchangeGateway` contract does not implement a constructor which disables the initializer for the implementation contract. As a result any malicious actor may claim ownership of the implementation contract for the `ExchangeGateway`.

In the past, this presented a significant issues since the malicious actor could use `upgradeToAndCall` on the implementation contract to self destruct the implementation itself and brick the proxy. However now it is not possible to use self destruct outside of the contract creation transaction.

Nonetheless it is a best practice to use a constructor with the disable initializers function to prevent any hostile takeovers of the implementation contract.

RECOMMENDATION

Implement a call to `_disableInitializers` in a constructor for the `ExchangeGateway` contract.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/commit/72a440edf99d1174ee4803b9c6ae6870e3429157>

LOW **L-03**

RESOLVED

Standalone OCO Orders Not Supported

CATEGORY	Logical Error
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

According to the Nekuti documentation (<https://www.nekuti.com/docs/orders/#one-cancels-the-other-oco>), standalone OCO orders should be possible without requiring a trigger from an OTO order. This would be useful if a user already has an active position, and would like to submit a take-profit and a stop-loss together, such that hitting one leg cancels the other.

Based on the current validation in the Sequencer, the first order must be OTO and all other orders in the group would be OCO:

```
@IsValidOtocoTrigger()
@IsBoolean()
@IsOptional()
@TransformDefault(false)
@ApiProperty({
  description:
    'Set if this order should be the OTO order in a new OTOCO group for triggering stop-loss/take-
    profit orders (precludes otocoGroup)',
  default: false,
  required: false,
})
readonly otocoTrigger: boolean = false;
```

RECOMMENDATION

Clearly document this to users or work with Nekuti to allow for standalone OCO orders.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/sequencer/commit/3c7b44ed27cfab7880a9d3608e2d6f0413e2fa03>

LOW **L-04**

ACKNOWLEDGED

Potential Manipulation of Funding Rate via Strategic Order Placement

CATEGORY Validation
LOCATION `funding.service.ts`
REVIEW Main Review

DESCRIPTION

The funding rate calculation is based on the mid price, which is the average of the best bid and best ask prices from the order book. This process involves three key functions: `updateFundingBasis`, `updateFunding` and `chargeFunding`. Each of these functions run on specific schedules as defined in the `DEFAULT_CONFIG`. An attacker could manipulate the funding rate by strategically placing and canceling large orders to influence the mid price, but they would need to do this multiple times within an hour to significantly skew the final funding rate. Below, we detail the roles of these functions, their execution frequencies, and how an attacker might exploit this setup.

Key functions and their schedules

1. `updateFundingBasis`

- This function calculates the funding basis, which is the difference between the mid price (derived from the order book) and the mark price (from an oracle). It records this basis to reflect real-time market conditions. Runs every 5 seconds, as specified by `basisSchedule`: `CronExpression.EVERY_5_SECONDS` in the `DEFAULT_CONFIG`. This frequent execution ensures the system captures a series of basis snapshots throughout the hour. The basis values collected here are aggregated later to determine the funding rate.

2. `updateFunding`

- This function aggregates the basis data collected by `updateFundingBasis` over the previous hour and calculates the funding rate to be applied. Runs hourly, specifically every 3 seconds within the first 15 seconds of each hour, as defined by `hourlyFundingSchedule`: `'0-15/3 0 * * * *'` in the `DEFAULT_CONFIG`. For example, it executes at 00:00:00, 00:00:03, 00:00:06, etc., up to 00:00:15. It finalizes the funding rate based on the hour's basis data, preparing it for application to traders' positions.

3. `chargeFunding`

- This function applies the funding rate calculated by `updateFunding` to the trading engine, adjusting traders' balances accordingly. Also runs hourly, within the same 15-second window at the start of each hour (aligned with `updateFunding`), as per the

L-04 Potential Manipulation of Funding Rate via Strategic Order Placement **CONTINUED**

hourlyFundingSchedule. It essentially executes the financial impact of the funding rate on all open positions.

Mid Price Calculation

The mid price is computed by the listMidPriceByProductIds method in the OrderRepository class, averaging the best bid and best ask prices from open orders:

```
listMidPriceByProductIds(productIds: bigint[], tx: TxCtx) {  
  return tx.trx  
    .selectFrom(this.relName)  
    .select([  
      'productId',  
      (eb) => {  
        const bestBid = eb.fn.max('price').filterWhere('side', '=', OrderSide.BUY);  
        const bestAsk = eb.fn.min('price').filterWhere('side', '=', OrderSide.SELL);  
        return eb  
          .case()  
          .when(eb.or([eb(bestBid, 'is', null), eb(bestAsk, 'is', null)]))  
          .then(0)  
          .else(eb(eb.parens(bestBid, '+', bestAsk), '/', eb.val(2).$castTo<string>()))  
          .end()  
          .as('midPrice');  
      },  
    ])  
    .where('productId', 'in', productIds)  
    .where('status', 'in', OPEN_ORDER_STATUSES)  
    .groupBy('productId')  
    .execute();  
}
```

This method takes a snapshot of the order book at the moment it's called, making it sensitive to sudden changes in the best bid or ask prices.

How an attacker could manipulate the funding rate?

An attacker could exploit this system by:

- Placing a large buy or sell order just before an `updateFundingBasis` execution (every 5 seconds) to manipulate the mid price. A large buy order would raise the best bid, increasing the mid price, while a large sell order would lower the best ask, decreasing it.
- Since `updateFundingBasis` runs every 5 seconds (12 times per minute, or 720 times per hour), the attacker would need to repeat this process multiple times within the hour to

L-04 Potential Manipulation of Funding Rate via Strategic Order Placement **CONTINUED**

meaningfully skew the basis data. The funding rate, calculated by `updateFunding`, is an aggregate of these basis values, so a single manipulation would have limited impact.

- After each `updateFundingBasis` snapshot, the attacker could cancel the order to avoid execution, minimizing financial risk while still influencing the basis.

Impact on traders

A manipulated funding rate could:

- Benefit the attacker: A higher rate might increase payments from short holders to long holders, favoring an attacker with a long position. A lower rate could reduce payments for an attacker with a short position.
- Harm other users: Unaware traders could face unexpected costs or reduced profits due to the artificial funding rate, undermining the system's fairness.

While the frequent basis updates (every 5 seconds) dilute the effect of a single manipulation, a persistent attacker targeting multiple snapshots could still shift the funding rate noticeably.

— RECOMMENDATION

To protect the funding rate calculation from this type of manipulation, consider introducing randomness into the timing of these samples, making it difficult for an attacker to predict when their order will influence the calculation. Third, establish monitoring to detect suspicious patterns, such as large orders placed and canceled in sync with known cron job schedules, enabling proactive responses to potential attacks. Fourth, enforce price bands that cap how much the mid price can deviate from recent trends or external reference prices, rejecting outliers that suggest manipulation.

— RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

Race Condition in Signer Linking Allows Exceeding Quota

CATEGORY	Race Condition
LOCATION	linked-signer.service.ts
REVIEW	Main Review

— DESCRIPTION

A race condition exists in the `linkSigner` method of `linked-signer.service.ts`, allowing an account to exceed its intended signer linking quota. The method first checks the number of linked signers for an account within a specified period using the `countByAccountLinkedWithinXDays` function, then proceeds to insert a new signer if the count is below the limit. In a concurrent execution scenario, two requests from the same account, each attempting to link a distinct signer, can perform this check at the same time. If both requests evaluate the count before either inserts a new signer, they may both find the count below the limit and proceed to add their signers. Although a uniqueness constraint prevents the same signer address from being linked twice, it does not prevent different signers from being added simultaneously. This flaw enables the account to exceed the quota, bypassing the rate-limiting mechanism designed to control signer linking.

— RECOMMENDATION

To resolve this issue, the period-limit check and signer insertion should be made atomic to eliminate concurrency risks. A practical solution is to wrap the operation in a database transaction with a locking mechanism that serializes access for a given account.

— RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

64-byte Signatures Are Rejected

CATEGORY Validation
LOCATION signature.service.ts
REVIEW Main Review

DESCRIPTION

Some wallets allow creating EIP-2098 "compact" signatures: 64 bytes signatures where the recovery bit (`v`) is packed into the top bit of `s` . The sequencer's current validator allows only legacy 65-byte `rllsllv` signatures:

```
// Verifies 'signature' is signed by 'signer' using 'message' of type 'primaryType'.
@AsyncMetricTimer(SignatureMetric.VERIFY_TIME)
async verifySignature(
  signature: Hash,
  signer: Address,
  message: Record<string, unknown>,
  primaryType: SignatureType
) {
  try {
    if (size(signature) !== SIGNATURE_LENGTH) { // export const SIGNATURE_LENGTH = 65;
      throw new SignatureError('Signature must be ${SIGNATURE_LENGTH} bytes');
    }
    return await verifyTypedData({
      address: signer,
      domain: this.rpcService.domainType,
      types: this.signTypedDataTypes,
      primaryType,
      message,
      signature,
    });
  } catch (err) {
    this.logger.error(getLogContext({ signer, signature }), 'Bad signature detected. Could not verify ${err}');
    return false;
  }
}
```

L-09 64-byte Signatures Are Rejected

CONTINUED

When a user submits an order or signer link produced by a modern wallet the HTTP API returns `401 Unauthorized Bad signature.`

Since the contracts does accept 64-byte signatures (`OpenZeppelin ECDSA.tryRecover()`) this check is an unnecessary blocker.

— RECOMMENDATION

Consider accepting both 64 and 65-byte signatures.

— RESOLUTION

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

Unbounded Sub-account Creation

CATEGORY Validation
LOCATION subaccount.service.ts
REVIEW Main Review

DESCRIPTION

Inside `SubaccountService.verifySubaccountAllowSigners()` the comment `// TODO: Cap the # of subaccounts a user can have` acknowledges that the system imposes no upper limit on how many sub-accounts a user may register. A determined user can therefore script the creation of tens of thousands of sub-accounts at negligible cost, each row occupies multiple tables. This unchecked growth inflates database size, bloats pagination queries and degrades every endpoint that filters by `account`.

RECOMMENDATION

Enforce a hard per-account quota inside the same transaction that inserts a new sub-account. The easiest approach is a serialized count check:

```
await tx.trx.selectFrom('account_lock')
  .where('address', '=', account)
  .forUpdate() // serialize concurrent sign-ups
  .execute();
const { count } = await tx.trx
  .selectFrom('subaccount')
  .select(sql`COUNT(*)`.as('count'))
  .where('account', '=', account)
  .executeTakeFirstOrThrow();
const MAX_SUBACCOUNTS = 10; // set by config
if (Number(count) >= MAX_SUBACCOUNTS) {
  throw new TooManyRequestsException('Maximum of ${MAX_SUBACCOUNTS} sub-accounts reached');
}
await subaccountRepository.create({ name, account, ... }, tx);
```

RESOLUTION

Status: Acknowledged.

LOW **L-11**

ACKNOWLEDGED

Precision Rounding in PennyJar Dust Metrics

CATEGORY Warning
LOCATION engine-vacuum.worker.ts
REVIEW Main Review

DESCRIPTION

In `engine-vacuum.worker.ts`, the PennyJar dust balances (`usage.total`, a `bigint`) are passed to Datadog via `Number(usage.total)`. While each individual “dust” amount is typically well below JavaScript’s safe-integer threshold ($2^{53} - 1$), over time the accumulated dust can exceed ~0.009 token (for 18-decimal assets) or millions of units for lower-decimal tokens. Casting a large `bigint` to a `JS number` silently rounds off low-order bits, causing metric spikes or drops that do not reflect the true balance, hiding real dust accumulation or triggering false alerts.

RECOMMENDATION

Consider introducing a `gaugeBigInt` helper in the metrics layer that accepts a `bigint` and only converts-to-number after safe scaling (e.g. divide by `10^decimals` to human units, ensuring the result $< 2^{53}$).

Replace all occurrences of

```
this.metricService.gauge(EngineMetric.PENNYJAR_BALANCE, Number(usage.total), { token: token.name });
```

with:

```
this.metricService.gaugeBigInt(EngineMetric.PENNYJAR_BALANCE, usage.total, { token: token.name });
```

Enforce via a lint rule (e.g. “`no-bigint-to-number`”) that `Number(bigint)` is disallowed outside of controlled scaling logic.

RESOLUTION

Status: Acknowledged.

LOW **L-13**

RESOLVED

Assymetrical Validations For Liquidations

CATEGORY DoS
LOCATION execution.handler.ts
REVIEW Main Review

DESCRIPTION

Function `relayLiquidations` is missing multiple validations that are performed within `liquidateSubaccount` on-chain, which may result in relaying a liquidation that will revert and DoS the entire batch of actions being processed.

Some of the missing validations include:

- `products.length == openPositions`
- `cost > 0`
- `position size + sizeDelta == 0`
- Liquidator has enough balance

RECOMMENDATION

Ensure the validations off-chain are symmetrical with the validations on-chain to prevent actions being relayed that will revert.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/sequencer/commit/cc9329a299da089d5974ac209380fb3e35de0488>

INFORMATIONAL I-01

RESOLVED

Non-standard Rate-Limit Headers

CATEGORY	Validation
LOCATION	rate-limit-account.service.ts
REVIEW	Main Review

DESCRIPTION

The sequencer uses custom rate-limit headers that differ from the emerging industry standard (IETF draft):

- IP-based limiter (limits by internet address):

```
response.headers({  
  'Retry-After' : retrySec,  
  'X-RateLimit-Limit' : total,  
  'X-RateLimit-Remaining': remaining,  
});
```

- Account-based limiter (limits by user account):

```
res.setHeader('X-RateLimit-Account-Retry-After', retrySec);  
res.setHeader('X-RateLimit-Account-Limit', total);  
res.setHeader('X-RateLimit-Account-Remaining', remaining);
```

The standard expects RateLimit-Limit, RateLimit-Remaining and RateLimit-Reset instead. These headers don't follow this pattern.

RECOMMENDATION

Switch to the standard headers following the IETF draft - <https://datatracker.ietf.org/doc/html/draft-ietf-httpapi-ratelimit-headers-08>:

```
response.headers({  
  'RateLimit-Limit' : total,  
  'RateLimit-Remaining' : remaining,  
  'RateLimit-Reset' : Math.ceil(retrySec),  
});
```

RESOLUTION

Status: Resolved.

I-01 Non-standard Rate-Limit Headers

CONTINUED

<https://github.com/meridianxyz/sequencer/commit/df24340f9f220df20151aabc6a11f468d604bc9a>

INFORMATIONAL **I-01.2**

RESOLVED

Price Feed With Non-8 Decimal Exponents

CATEGORY Warning
LOCATION PerpEngine.sol
REVIEW Main Review

DESCRIPTION

Pyth price feeds that have non-8 Decimal exponents such as PEPE/USD (-10 exponent) should not be registered on-chain, as the Sequencer will not accept prices from non-8 decimal exponent feeds:

```
if (priceFeed.exponent !== -8) {  
  this.logAndThrow('Non -8 exponent prices not supported', getLogContext(priceFeed));  
}
```

RECOMMENDATION

Clearly document that such feeds should not be registered.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/commit/39c750bce8c2f1a3c9278ef1aee406368dd28bc1>

INFORMATIONAL I-02

RESOLVED

Incorrect Return Property

CATEGORY Typo
LOCATION relayer.service.ts: 201
REVIEW Main Review

DESCRIPTION

The object returned from function `capActionsUsingEstimatedGasUsage` is `{ gsEstimate: sumGasEstimate, cappedActions: actions }`; but property `gasEstimate` is expected rather than `gsEstimate` (typo): `const { gasEstimate, cappedActions } = this.capActionsUsingEstimatedGasUsage(actionsBatch, gasLimit);` Consequently, `gasEstimate` will be set to undefined and the log context will use that inaccurate `undefined` value.

RECOMMENDATION

Within function `capActionsUsingEstimatedGasUsage`, change the return object key from `gsEstimate` to `gasEstimate`

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/sequencer/commit/b2bc88204bd0e031123f2cc156e0c9c4b26af13e>

INFORMATIONAL I-02.2

ACKNOWLEDGED

Facets Entry-points Lack a “gateway-only” Guard

CATEGORY	Warning
LOCATION	CollateralManager.sol, ActionHandler.sol
REVIEW	Main Review

DESCRIPTION

`CollateralManager` and `ActionHandler` contracts keep their main functions (`deposit` , `depositUsd` , `processActions` , etc.) marked as external, so they can be invoked directly through their own contract addresses instead of through the `ExchangeGateway` . A direct call executes in the facet's separate storage context. This does not threaten core solvency but it can trap a user's tokens in the facet address and emit misleading events, degrading UX and supportability.

RECOMMENDATION

Gate every external function in delegate-only facets with a modifier that reverts unless the call originated from the proxy context, e.g.:

```
/// @dev Throws if not called via delegatecall from the proxy/gateway.
modifier onlyGateway() {
    require(msg.sender == address(this), "Facet: direct call disallowed");
    _;
}
```

Apply the `onlyGateway` modifier to `CollateralManager.deposit` , `depositOnBehalf` , `depositUsd` , `finalizeWithdraw` , `claimFees` and to `ActionHandler.processActions` .

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03.2

RESOLVED

Documentation Typos

CATEGORY	Informational
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The IExchangeConfig interface has some minor comment typos:

- `updatePythLazer` description: "contract uses for price verification" → should be "used for price verification".
- `addDelegateDepositor`: "Grants the delegate depositor role tn an address" → "role to an address".
- `removeDelegateDepositor` param doc: "who will have their the role remove" → "their role removed".
- `updateDelegateDepositMaxBatchSize` param doc: "that can passed into depositOnBehalf" → "that can be passed".
- `removeEmergencyPauser` param doc: "will have their the role remove" → "their role removed".
- `updateToken` function doc: "amounts are token native to token" → "amounts are in the token's native units".

There is a typo on line 33 of `ICollateralManager`: "... value amountof USD..." should be corrected to "... value amount of USD..."

RECOMMENDATION

Update the NatSpec accordingly.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/200>

INFORMATIONAL I-04.2

ACKNOWLEDGED

depositOnBehalf Can Be DoS'ed

CATEGORY	Frontrunning
LOCATION	https://github.com/GuardianOrg/contractsmeridian-sequencer1-team2/blob/0c158ff08758dc60df2d2d63c2907edd6fe7fe58/src/CollateralManager.sol#L214
REVIEW	Main Review

— DESCRIPTION

The depositor can deposit on behalf of multiple accounts at once, but all deposit requests will fail if a single deposit fails during the loop.

The same account/subaccount pair can only have one pending deposit at a time, which is enforced in the `_verifyDeposit` function. A malicious receiver (account) can front-run the depositor's call and create a pending deposit using the same subaccount, causing all deposits in the call to revert.

— RECOMMENDATION

There is no incentive for a receiver to intentionally block deposits that would ultimately benefit them. However, it's important to be aware that this situation can also arise unintentionally, without any malicious activity, and would result in unnecessary gas costs for the depositor.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-05.2

RESOLVED

Consider Native Transfer During claimFees

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/contractsmeridian-sequencer1-team2/blob/0c158ff08758dc60df2d2d63c2907edd6fe7fe58/src/CollateralManager.sol#L67
REVIEW	Main Review

— DESCRIPTION

The withdrawal flow unwraps WUSDe and transfers the native token to the user if `withdrawToken == usdToken`.

However, this is not the case in the `claimFees` function. The `feeToken` is transferred to the fee collector without unwrapping, even if `feeToken == usdToken`.

— RECOMMENDATION

Clearly document this behavior, or consider unwrapping WUSDe and transferring the native token if `feeToken == usdToken` for symmetrical behavior.

— RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/201>

INFORMATIONAL I-06.2

RESOLVED

wUSDe Deposits Withdrawn As Native

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contractsmeridian-sequencer1-team2/blob/0c158ff08758dc60df2d2d63c2907edd6fe7fe58/src/CollateralManager.sol#L158
REVIEW	Main Review

— DESCRIPTION

Users can deposit native USDe using the `depositUsd` function, or they can directly deposit wrapped USDe using the regular `deposit` function. Both types of deposits are credited to the same `usdToken` balance of the user.

However, all `usdToken` withdrawals are executed as native USDe withdrawals, regardless of whether the original deposit was in native or wrapped form.

— RECOMMENDATION

Clearly document this behavior, or consider allowing users to choose whether they want to receive native or wrapped tokens during withdrawal.

— RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/contracts/pull/208>

INFORMATIONAL I-07.2

RESOLVED

Referral Gaming

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

Currently there is no implementation for the handling of the `AccountReferred` event on the backend.

However due to the logic of the `AccountReferred` event emission in the Ethereum contracts the future implementation of this referral handling may be exposed to a certain level of gameability.

The event is emitted for an entire account, presumably updating the accounts referral code when a deposit is made. However there is no validation that this is the first deposit for the account and that the associated referral code is indeed the initial referrer.

As a result traders can make follow-up deposits to re-write their referral code and remove the referral incentive that was promised to the individual who referred the user in the first place. Furthermore, the `depositOnBehalf` function now allows for the caller to potentially unexpectedly change the target user's referral code, although deposit on behalf actions are permissioned and thus presumably trusted.

RECOMMENDATION

Be aware of this behavior for the `AccountReferred` and consider if this is acceptable for the desired implementation of account referrals.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-08.2

RESOLVED

Lacking Compatibility With Non-USD Tokens

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The deposit flow in the Ethereum contracts supports non-usd tokens for deposits, however the rest of the Ethereum contract system does not support non-usd tokens as collateral.

For example, the liquidation flow does not zero out non-usd token collateral when an account is liquidated. Furthermore, the `_chargeFee` function during order matching would not correctly be able to charge fees and revert for accounts with alternative collateral tokens.

Non-usd tokens can technically be added with the `addToken` function, though a comment in the liquidation flow states that only USD tokens will be supported on launch.

RECOMMENDATION

Be aware of this incompatibility in the smart contracts and be sure to not add any non-usd tokens as collateral before the liquidation flow, fee charging and other areas have been updated.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-09.2

ACKNOWLEDGED

Pure Modifier Used On Storage Loads

CATEGORY	Logical Error
LOCATION	ExchangeGateway.sol
REVIEW	Main Review

DESCRIPTION

The function `getToken(bytes32 tokenName)` is declared `pure`, yet it invokes `Token.load(tokenName)`, which loads from contract storage. Solidity enforces that `pure` functions cannot access storage, and although the contracts compile this may be confusing and asymmetrical with other functions in the ExchangeGateway.

RECOMMENDATION

Consider updating the function signatures for `getToken` and `getFeeSchedule` to external `view` instead of `pure`.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

ACKNOWLEDGED

Number Of Positions Opened Are Not Limited At Smart Contract Level

CATEGORY	Validation
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The sequencer enforces a per-account cap on simultaneously open positions via the off-chain call made during bootstrap:

```
// engine-listen.worker.ts – line ~70
await
this.engineService.setMaxPositionsLimit(this.configService.sequencer.account.maxPositions);
// sequencer.dto.ts
account: {
  linkSignerLimitInPeriod: 5,
  linkSignerLimitPeriodDays: 7,
  linkSignerExpiryDays: 30,
  maxPositions: 50,
},
```

That limit lives only in the back-end as there is no restriction at smart contract level.

RECOMMENDATION

Consider enforcing a cap also at smart contract level. For example, add a state constant (e.g. `uint256 public immutable MAX_POSITIONS = 50;`) and insert a guard before every code path that calls `_openPosition`, `_increasePosition` or creates a fresh `Position` struct:

```
require(
  subaccounts[_subId].openPositionCount < MAX_POSITIONS,
  "MAX_POSITIONS_EXCEEDED"
);
```

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

RESOLVED

Signer Typo

CATEGORY	Typo
LOCATION	linked-signer.service.ts: 85
REVIEW	Main Review

DESCRIPTION

The log within `linkSigner` is currently `this.logger.debug(logContext, 'Attempting to link singer');` where "singer" should be "signer".

RECOMMENDATION

Correct the typo.

RESOLUTION

Status: Resolved.

<https://github.com/meridianxyz/sequencer/pull/764>

INFORMATIONAL I-06

ACKNOWLEDGED

Stale Oracle Prices Can Be Used For Funding Calculations

CATEGORY Warning
LOCATION funding.service.ts
REVIEW Main Review

DESCRIPTION

In the `funding.service.ts` file, the `getMarkPrices` method retrieves mark prices from Pyth to use them in the funding calculations, but it does not adequately handle cases where these prices are stale. A price is considered stale if its age exceeds the defined `ORACLE_PRICE_MAX_AGE_MS`. In such cases, the current implementation logs a warning via `this.logger.warn(getLogContext({ ticker: product.ticker, priceAge }), 'Stale cached mark price');` but proceeds to return the outdated value with `return { product, price: Gwei.toGwei(markPrice.price), timestamp: markPrice.timestamp };`. This means funding calculations may rely on inaccurate or obsolete data.

RECOMMENDATION

Consider updating the `funding.service.ts` file to robustly handle stale oracle prices by introducing a configurable option that rejects prices older than `ORACLE_PRICE_MAX_AGE_MS` and for example, simply consider only the mid price for the new funding rate calculation for that specific asset where the price received is stale.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-07**

ACKNOWLEDGED

Information Leakage About Orders

CATEGORY Warning
LOCATION order.controller.ts
REVIEW Main Review

— DESCRIPTION

The API allows for anyone to call endpoints such as /order/trade, /order/fill, and /order which allows a malicious user to view the orders, fills, trades for any user of Ethereum. This information can be used to figure out whether a specific trader is using hidden orders and there is more dormant liquidity waiting and a variety of other trading advantages that come from knowing if a specific account is creating orders.

— RECOMMENDATION

Clearly document whether this is intended behavior or consider using an AuthGuard

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-08

ACKNOWLEDGED

Whitelist Enabled During Market Activity

CATEGORY Logical Error
LOCATION whitelist.service.ts
REVIEW Main Review

DESCRIPTION

The Config service has the ability to toggle whether whitelisting is enabled:

```
async isWhitelisted(account: Address) {  
  if (!this.configService.sequencer.whitelist.enabled) {  
    return true;  
  }  
  const whitelist = await this.getWhitelist();  
  return whitelist.has(account);  
}
```

A situation may arise where there is ongoing trading activity, whitelisting validation is enabled without adding users actively trading, and then those users are prevented from managing their orders — ultimately leading to losing trades and fund loss.

RECOMMENDATION

Clearly document this behavior and how the whitelisting will be operated.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-09

ACKNOWLEDGED

OCO Orders Created After Trigger

CATEGORY Logical Error
LOCATION Global
REVIEW Main Review

DESCRIPTION

After the OTO trigger is executed, all the orders within the group become active OCO orders. In most cases users can keep submitting orders to the group and they will be treated as OCO's within that group, and cancel all other orders if executed.

Note that there are some exceptions to this, such that with TimeInForce IOC for the trigger order, if the trigger was cancelled due to 'UnfilledImmediateOrCancel', following orders will be rejected with 'TriggerCanceled', rather than just being 'Pending'. For TimeInForce FOK for the trigger order, even if the trigger was outright cancelled upon submission due to 'UnfilledFillOrKill', following conditional orders for that group will indeed be 'Pending' OCO orders, which shows that there is some inconsistency with Nekuti's handling of filled/cancelled trigger orders.

RECOMMENDATION

Clearly document this behavior to users.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-10**

ACKNOWLEDGED

Excluded Signers Should Be Validated

CATEGORY	Warning
LOCATION	linked-signer.service.ts
REVIEW	Main Review

— DESCRIPTION

Within function `linkSigner`, a series of validations are performed on the provided user parameters. It may also be prudent to verify the intended linked signer against the `excludedSigners` mapping on-chain, to prevent any potential cases where a `InvalidSigner` is thrown and DoS's a batch.

— RECOMMENDATION

Consider also validating that the signer is not part of `excludedSigners` on the contracts

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-11**

ACKNOWLEDGED

Action Batch DoS Resiliency

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Within function `capToIndexBeforeActionId`, if the first action in the batch is the one triggering an error on-chain then an error, then no actions will be present within the `simulatedActions` array. Every subsequent call to `batchActions()` will pick up that same invalid transaction, re-simulate and error again. Consequently, one invalid entry will block the Relay from operating.

Furthermore, there may be instances where different workers in the Sequencer crash and on restart already processed actions are attempted to be relayed. This has happened more than once within the local development environment, although is difficult to directly reproduce.

— RECOMMENDATION

Consider having a privileged ability to mark certain actions a different status and move on to subsequent actions. The order of actions is extremely important, so care must be taken when skipping.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-12**

ACKNOWLEDGED

Inconsistent Subaccount Throw Handling

CATEGORY Best Practices
LOCATION linked-signer.service.ts
REVIEW Main Review

— DESCRIPTION

Within function `revokeSigner` does not specify an `orThrow` parameter: `const subaccount = await this.subaccountService.getByExtId(subaccountExtId, tx);`

In contrast, function `linkSigner` does specify the parameter when querying the database: `await this.subaccountRepository.getByExtId(subaccountExtId, tx, { orThrow: true });`

— RECOMMENDATION

Consider making the database queries to get the subaccount by external ID symmetric.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-13

ACKNOWLEDGED

Leaves Quantity Naming

CATEGORY Best Practices
LOCATION execution.handler.ts
REVIEW Main Review

DESCRIPTION

After each fill, the `leavesQuantity` is set to the size of the new position. This naming can be confusing since typical FIX nomenclature has 'leaves' referring to the unfilled portion of an order rather than what was filled.

RECOMMENDATION

Document that the column refers to position size after the current fill.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-14**

ACKNOWLEDGED

Table-wide EXCLUSIVE Lock On relayerAction

CATEGORY	Warning
LOCATION	relayer-action.repository.ts
REVIEW	Main Review

DESCRIPTION

This is the normal operational flow for `RelayerActionRepository.create` and where it actually sits in the pipeline:

- API layer – Every signed endpoint (`/order/match`, `/withdraw/initiate`, ...) ends with `RelayerActionService.enqueue()`.
- Engine listener – The engine-listen module invokes the same `enqueue()` for maker/taker fills, funding deltas, etc.
- Relay worker – Periodically reads rows where `status='PENDING'`, bundles them, and calls the Solidity `processActions`.

Thus every state-changing event passes through `RelayerActionRepository.create()` at line 9 of `relayer-action.repository.ts`:

```
async create(action: Insertable<RelayerAction>, tx: TxCtx) {  
  // Table-wide lock  
  await this.dbService.lockTable(this.relName, 'EXCLUSIVE', tx.trx);  
  return tx.trx  
    .insertInto(this.relName)  
    .values(action)  
    .returning(['id', 'type'])  
    .executeTakeFirstOrThrow();  
}
```

However, `LOCK TABLE ... IN EXCLUSIVE MODE` tells PostgreSQL: "no other session may `SELECT ... FOR UPDATE`, `INSERT`, `UPDATE` or `DELETE` until I commit." The lock is therefore acquired for every single row insert, both in `create()` and `createBatch()`. Under normal traffic this serialises all producers into a single-threaded critical section. Under burst traffic or a deliberate spam attack, legitimate producers (especially the Engine listener) queue behind the attacker and stall, so the relay never receives fresh actions to send on-chain.

I-14 Table-wide EXCLUSIVE Lock On relayAction

CONTINUED

This could cause **PENDING** rows to pile up as relay's batcher has nothing new to process because the listener cannot insert, order-fill/withdraw/liquidation confirmations to freeze, front-end shows them as "pending" indefinitely and funding updates and liquidations miss their on-chain windows, potentially creating bad debt.

— RECOMMENDATION

Merely informational.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-15

ACKNOWLEDGED

Dockerfile Warning

CATEGORY	Warning
LOCATION	Dockerfile
REVIEW	Main Review

DESCRIPTION

The sequencer's Dockerfile does not specify a `USER` to run commands as, therefore the `root` user will be used by default. This poses several security risks, which can lead to privilege escalation if the container is compromised.

Furthermore, there is no `HEALTHCHECK` instruction present in the Dockerfile. This instruction helps keep track of the status of the containerized application, and can improve reliability and availability.

RECOMMENDATION

Specify a `USER` in the Dockerfile other than `root` and implement a `HEALTHCHECK` instruction in the Dockerfile.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.